PDFSage Security Assessment Report

Penetration test, remediation and re-test. CSSE 606, University of Karachi. Assessed 25 Oct 2026.

1. Executive summary

A black-box and code-review assessment of PDFSage found **14 weaknesses out of 16 checks**, including **5 High** severity. The worst problems were: no authentication, no protection against cross-site (CSRF) and DNS-rebinding attacks, no limits on PDF size/pages, and shared state that let any device on the network use the first user's document. All were fixed and the same 16 attack checks were re-run: **0 remain**. The 8 functional tests plus 3 new security tests (11 total) pass, Bandit static analysis reports no issues, and the dependency audit found two outdated packages (see section 5).

2. Scope and method

Target: Flask app, Ollama client, PDF/OCR pipeline, browser UI. **Attacker models:** (a) another device on the same Wi-Fi, (b) a malicious website opened in the victim's browser, (c) a malicious PDF, (d) a user sending malformed API requests. **Tools:** scripted attacks (tests/pentest.py), Bandit, pip-audit, manual code review. Every finding below was reproduced with a script, not assumed.

3. Findings and re-test results

ID	Sev.	Finding	Evidence (before fix)	After fix
F01	High	No authentication: any LAN host can read/use the app	LAN client GET /api/status -> 200	Fixed
F02	High	No Host-header validation (DNS-rebinding)	Host: evil.example -> 200	Fixed
F03	High	CSRF: cross-site page can upload/replace the document	Origin evil.example upload -> 200	Fixed
F04	Medium	CSRF: cross-site request can change the model	-> 200	Fixed
F05	Medium	Malformed JSON / bad numbers cause HTTP 500	status codes [500, 500, 500]	Fixed
F06	Medium	Unbounded 'n' goes straight into the LLM prompt (resource abuse)	n=10^9 accepted into prompt	Fixed
F07	Medium	Upload size limit	60 MB upload -> 413	Not vulnerable
F08	High	No page cap: huge/scanned PDFs exhaust CPU (OCR on every blank page)	150-page PDF -> 200; OCR cost ~0.12s/page, so 1000 scanned pages is roughly 2 min of CPU	Fixed
F09	Medium	Dense TF-IDF matrix: memory grows with chunks x vocabulary	2425 chunks -> 582 MB matrix	Fixed
F10	Low	Internal exception text leaked to client	error = Could not read PDF: Failed to open stream	Fixed
F11	Medium	Missing security headers (CSP, clickjacking, MIME sniffing)	missing: ['Content-Security-Policy', 'X-Frame-Options', 'X-Content-Type-Options']	Fixed
F12	High	Shared global state: another user reads the first user's document	2nd client chat -> 200	Fixed
F13	Medium	Indirect prompt injection: document text is not fenced from instructions	injected text reaches the prompt as plain instructions	Fixed
F14	Low	Arbitrary model-name string accepted	-> 200	Fixed
F15	Low	No concurrency guard: parallel requests pile up on the single local LLM	no lock	Fixed

ID	Sev.	Finding	Evidence (before fix)	After fix
F16	High	DOM XSS via filename / LLM output	reviewed: model/PDF text is inserted with textContent; filename is escaped	Not vulnerable

4. Attack scenarios and fixes

F01 - No authentication: any LAN host can read/use the app (High)

Attack scenario: Attacker on the same Wi-Fi uses your uploaded PDF, reads its text via chat, and burns your CPU.
Fix: Remote access is refused unless PDFSAGE_TOKEN is set; token is checked with constant-time compare; HttpOnly + SameSite=Strict cookie.

F02 - No Host-header validation (DNS-rebinding) (High)

Attack scenario: A malicious website re-points its DNS name at 127.0.0.1 and then reads your app as 'same origin'.
Fix: Host header must be localhost/127.0.0.1 or an explicit allow-list entry.

F03 - CSRF: cross-site page can upload/replace the document (High)

Attack scenario: A web page you visit silently submits a file to localhost:5000 and replaces your document.
Fix: POST requests whose Origin differs from the Host are rejected (403).

F04 - CSRF: cross-site request can change the model (Medium)

Attack scenario: A web page switches your model or breaks your session.
Fix: Same Origin check; model name validated by regex (F14).

F05 - Malformed JSON / bad numbers cause HTTP 500 (Medium)

Attack scenario: One malformed request crashes the endpoint (500) and fills logs.
Fix: body() and num() validate input: wrong types return 400, numbers fall back to defaults; global handler hides stack traces.

F06 - Unbounded 'n' goes straight into the LLM prompt (resource abuse) (Medium)

Attack scenario: Someone asks for a billion questions and keeps the local LLM busy.
Fix: n is clamped (questions 1-12, MCQs 1-8); chat question capped at 500 characters.

F08 - No page cap: huge/scanned PDFs exhaust CPU (OCR on every blank page) (High)

Attack scenario: A scanned 1000-page PDF ties up OCR for many minutes.
Fix: Page cap (default 100, env PDFSAGE_MAX_PAGES), encrypted PDFs refused, %PDF- signature check.

F09 - Dense TF-IDF matrix: memory grows with chunks x vocabulary (Medium)

Attack scenario: Large document needs hundreds of MB of RAM and can crash the app.
Fix: TF-IDF vocabulary capped at 4000 features and chunks capped at 600.

F10 - Internal exception text leaked to client (Low)

Attack scenario: Parser error text helps an attacker fingerprint the library.
Fix: Generic message to the user; details only in the server log.

F11 - Missing security headers (CSP, clickjacking, MIME sniffing) (Medium)

Attack scenario: Page can be framed for clickjacking; any injected script would run.
Fix: CSP (no inline script), X-Frame-Options DENY, nosniff, no-referrer, no-store on API. JS and CSS moved to /static.

F12 - Shared global state: another user reads the first user's document (High)

Attack scenario: Second person on the network reads the first person's document.
Fix: Covered by F01 plus a single-flight lock; the document is single-user by design.

F13 - Indirect prompt injection: document text is not fenced from instructions (Medium)

Attack scenario: A crafted PDF tells the model to ignore the task and lie in answers.
Fix: Document text is wrapped in tags and the model is told it is untrusted data.

F14 - Arbitrary model-name string accepted (Low)

Attack scenario: Odd strings flow into the Ollama request.
Fix: Regex: letters, digits, . _ : / - only, max 64 characters.

F15 - No concurrency guard: parallel requests pile up on the single local LLM (Low)

Attack scenario: Ten parallel requests queue on one 3B model and freeze the UI.
Fix: Non-blocking lock; extra requests get 429 'model busy'.

5. Residual risks and recommendations

Plain HTTP on the LAN: The token and traffic can be sniffed on untrusted Wi-Fi. Use home/trusted Wi-Fi only, or put a TLS reverse proxy (Caddy, nginx) in front.

Prompt injection is mitigated, not solved: A hostile PDF can still sway what the model says. Impact is limited because the model has no tools and its output is shown as plain text, but answers from untrusted PDFs should be verified.

Malicious PDFs vs. the PDF parser: PyMuPDF/MuPDF is native C code. Keep it updated; for untrusted files run the app in a container or low-privilege account.

Ollama itself: Ollama listens on localhost:11434 by default. Never set OLLAMA_HOST=0.0.0.0 or open that port in the firewall; it has no authentication.

Dependency vulnerabilities: pip-audit flagged werkzeug 3.1.7 (fixed in 3.1.9) and urllib3 2.6.3 (fixed in 2.7.0). requirements.txt now requires the fixed versions: run pip install -U -r requirements.txt.

Flask development server: Fine for a personal tool; for shared use run behind waitress/gunicorn. Cookie is not marked Secure because the app uses HTTP.

'unsafe-inline' for styles: Kept because the UI uses a few inline style attributes; scripts are strictly self-hosted.

Model and package supply chain: Models (Ollama, Hugging Face) and pip packages are downloaded from the internet: install only from official sources and pin versions for the final submission.

Not tested: Real browser exploitation (done by code review and CSP), real Ollama model behaviour, Windows firewall configuration, and denial-of-service from many machines.

6. How to re-run

`python tests/pentest.py check` prints OK/VULN for all 16 checks. `python -m pytest -q` runs functional and security tests. `bandit -r app.py backend` and `pip-audit` cover code and dependencies.